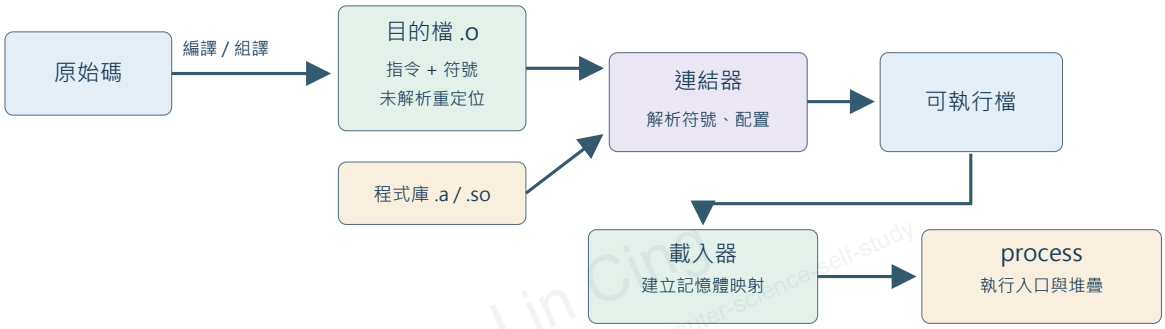
原始碼到機器狀態的五個邊界

編譯器將高階程式轉成組合語言或直接產生目的檔。組譯器（assembler）處理指令、標籤與組譯指示，產生機器碼、符號與重定位資訊。連結器（linker）解析跨檔案的名稱並配置位址；載入器建立行程的記憶體映射，CPU 再依指令集執行。



目的檔格式正確不代表程式能正確執行，連結成功也不代表呼叫慣例正確。仍須檢查指令欄位、符號綁定、重定位位址、記憶體區段權限，以及呼叫雙方約定的暫存器保存責任。

本章 MIPS 模型

採傳統 pre-Release-6 MIPS32：每條機器指令 4 bytes，branch／jump 有一個 delay slot，`jal` 把呼叫指令位址加 8 寫入 `$ra`。若教材採沒有 delay slot 的簡化模型，返回位址可能是 PC+4；兩種慣例不能混用。範例以nop明示slot，不假設load delay。SPIM需選相容的delayed-branch設定；它的syscall是教學介面，不是Linux MIPS syscall ABI。

位址運算採32-bit，整數結果需要時按mod2^32解讀。一般branch immediate先sign-extend再乘4，J-format target則由PC+4高4 bits拼接26-bit field與兩個0。本文簡化object與relocation模型會明示addend，不能當作完整ELF parser。

B.1 Assembly 每一行的不同意義

`addu $t0,$t1,$t2`是machine instruction。`.text`切換section、`.word`配置資料、`.align n`在本章assembler語法對齊到2^n bytes，這些directives不由CPU逐行執行。

`move $s0,$s1`可展開為`addu $s0,$s1,$zero`；`la $t0,label`可能展開成兩條甚至不同位址取得序列。Macro是文字/語法層展開，不建立runtime call frame。程式大小必須由展開後的指令、資料及padding計算，不能按source行數乘4。

直接用assembly適合boot、exception entry或需要精確ABI的低階邊界；效能優勢須依實際編碼與模型量測，不能由「較低階」推論一定較快。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.2 兩遍組譯的資料結構

第一遍維護每個section自己的location counter，並記錄symbol的section與offset。遇到label就綁定當前counter；遇到指令或資料就增加確切size；遇到alignment就向上取到合法邊界。

第二遍依相同展開與size規則編碼。已知同section的PC-relative reference可解出差值；仍需最終section base或外部symbol者產生relocation。兩遍若對pseudoinstruction長度的判斷不同，所有後續label都會錯；實作可先建立已展開的中間表示，避免各遍各猜一次。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Location counter 的實際追蹤

本例.text offset從0開始，每條顯示的指令均已是4-byte machine instruction。

```
.text
main: addiu $t0,$zero,0
loop: addiu $t0,$t0,1
      bne   $t0,$a0,loop
      nop
done: jr    $ra
      nop
.data
msg:  .asciiz "Hi"
.align 2
value: .word 0x1234abcd
```

事件	section offset之前	配置/對齊後
main指令	text0	text4
loop指令	text4	text8
bne、nop	text8	text16
done: jr、nop	text16	text24
msg及NUL	data0	data3
.align 2	data3	data4，補1 byte
value	data4	data8

Location counter 的實際追蹤 (續)

因此symbols為main=(text,0)、loop=(text,4)、done=(text,16)、msg=(data,0)、value=(data,4)。
Section base直到link才決定。

Branch encoding 與 relocation invariance

若text base=0x00400000，bne在0x00400008，loop在0x00400004。Immediate為

$$\frac{target-(PC+4)}{4}=\frac{0x00400004-0x0040000c}{4}=-2=0xfffe.$$

bne opcode5、rs=$t0=8$、$rt=$a0=4，完整word為

$$(5\ll 26)|(8\ll 21)|(4\ll 16)|0xfffe=0x1504fffe.$$

整個text搬移δ時，target與PC同加δ，差值不變。因此已知同section、相對位置不變的branch通常不需為單純整段搬移而修改。若linker改寫layout或target在其他section，就要重算並檢查signed16-bit範圍及4-byte對齊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

超出分支範圍時如何改寫

在本章有 delay slot 的模型中，假設 `beq $s0,$s1,far` 位於 0x00400000，且原 delay slot 為 `nop`；遠端標籤 `far=0x00430000`。分支位移是 (0x00430000−0x00400004)/4=49151，超過 signed 16-bit 的最大值 32767。

```
0x00400000  bne  $s0,$s1,skip
0x00400004  nop                 # branch delay slot
0x00400008  j    far
0x0040000c  nop                 # jump delay slot
0x00400010  skip:  ...          # 原來不跳轉的後續程式
```

相等時 bne 不成立，執行 slot 後由 j 到 far；不相等時執行 slot 後直接到 skip。bne 的位移是 (0x10−0x04)/4=3。此例要求遠端與 j 的 PC+4 在同一 256 MiB 區域；否則還須使用可達的跳板或間接跳躍。改寫後程式變長，後續位址與其他 reference 必須重新配置。

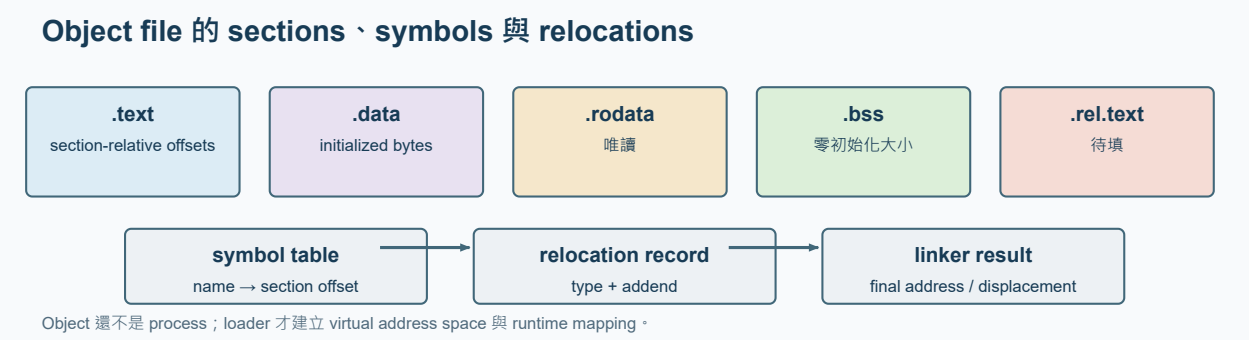
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Macro 展開也可能改變輸入

假設一個不安全的macro要把argument乘3，展開成 `addiu $t0,$zero,3; mult $arg,$t0; mflo $v0`。若呼叫的argument正是t0，且原值7，展開後第一條已把7覆寫成3，結果為9而非21。Macro作者必須明定clobbers，保證讀取原argument前不覆寫它，或使用不與arguments衝突的暫存策略。

重複展開內部label也需產生唯一名稱，否則可能重複定義或跳到另一份macro。Conditional assembly在組譯時選擇要輸出的code，不是CPU執行期的conditional branch；未輸出的分支沒有runtime成本，但配置與symbol table會隨條件改變。

Object file 不是記憶體快照



`.text`保存指令，`.data`保存有初值的可寫資料，`.rodata`保存唯讀常數，`.bss`描述應配置並清零的大小，通常不需在檔案中放同量的0 bytes。Symbol除了名稱還有section、value、binding與type。

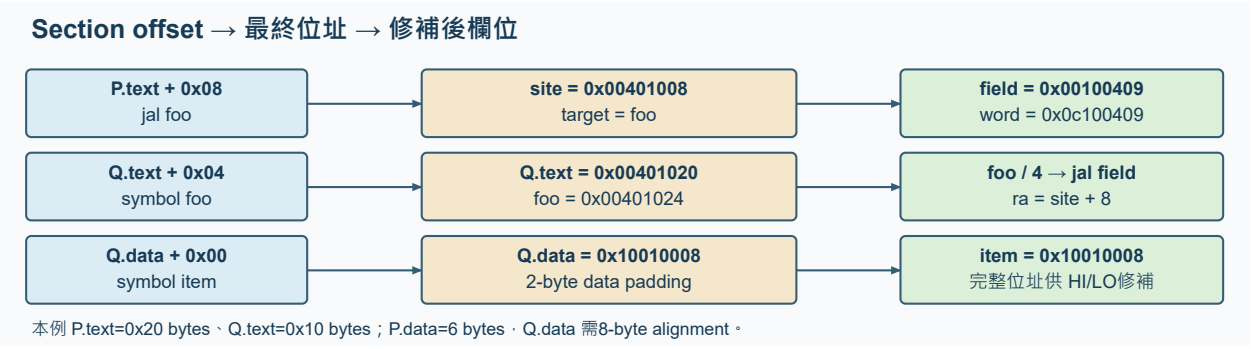
Relocation record至少要指出site、symbol、type及addend的取得方式。有的格式把addend放在record，有的編在原欄位；不能只看欄位0就猜「不需要修補」。Debug資訊也不是執行時一般load/store的資料。

Symbol resolution 的範圍

Local symbols只在各自object有效，所以兩個object可各有名為loop的local label。兩個同名strong global definitions通常是錯誤；undefined reference必須由合法definition、library或執行期綁定機制滿足。

Weak/common與symbol visibility依格式及工具鏈規則處理，不能泛稱linker任選一個。以下例子只用唯一strong definition，使每個reference有明確target；若不唯一，就先報錯，不能靠relocation補救名稱歧義。

B.3 合併兩個 objects 的 sections



明定P.text大小0x20、Q.text大小0x10；P.data大小6 bytes、Q.data大小4 bytes且需8-byte alignment。Text base=0x00401000，data base=0x10010000，按P、Q順序合併。

輸入section	最終base	範圍，含首不含尾
P.text	0x00401000	[0x00401000,0x00401020)
Q.text	0x00401020	[0x00401020,0x00401030)
P.data	0x10010000	[0x10010000,0x10010006)
padding	0x10010006	2 bytes
Q.data	0x10010008	[0x10010008,0x1001000c)

一個 jal relocation 的全部欄位

若Q中foo=(text,4)、item=(data,0)，則最終foo=0x00401024、item=0x10010008。每個symbol的offset加的是**自己輸入section的最終base**，不是一律加整個executable base。

P.text offset8有`jal foo`，site=0x00401008。Foo=0x00401024且aligned，PC+4=0x0040100c的高4 bits與target相同，所以可用J-format。

$$field = (0x00401024 \gg 2)\ \&\ 0x03ffffff = 0x00100409,$$

$$word = (3 \ll 26) | field = 0x0c100409.$$

CPU重建target為 $\{(PC+4)[31:28], field, 00\} = 0x00401024$。`jal`另把return address 0x00401010寫入ra，並先執行site+4的delay-slot nop。若target跨256 MiB區域，截26 bits不能修好；需合法的veneer或間接跳躍序列，不能默默截斷。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

High/low relocation 與 sign extension

對V=0x1234abcd，`lui/ori`採零延伸low：high=0x1234、low=0xabcd，$(high \ll 16) | low = V$。

若第二條用`addiu`，low立即數sign-extend，0xabcd代表−21555。因此high必須進位：

$$H = ((V + 0x8000) \gg 16)\ \&\ 0xffff = 0x1235,$$

$$(0x1235 \ll 16) - 21555 = 0x1234abcd.$$

把0x1234直接配addiu得到0x1233abcd，差0x10000。實際MIPS HI16/LO16配對relocation還要依ABI處理addend與pair關係；本例是完整最終值V已知後的算術，不將兩個獨立截半視為所有relocation的實作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Static archive 的一次掃描例

明定教學linker由左至右處理檔案，object必納入；遇archive時，反覆抽出能滿足當前undefined symbols的members直到該archive不再有新需求，之後不自動重掃前面的archive。

Main.o定義main、引用f；libF.a的F.o定義f、引用g；libG.a的G.o定義g。`main.o libF.a libG.a`的未解集合依序為{f}、{g}、{}，成功。

`main.o libG.a libF.a`先在libG時只有f未解，無member可抽出；libF再抽F.o後留下g，最終失敗。這不是C語言語意而是明定archive選取規則。實際工具鏈可有group/rescan選項，不能把這個單向模型當作所有linker唯一行為。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

動態連結與 GOT 間接引用

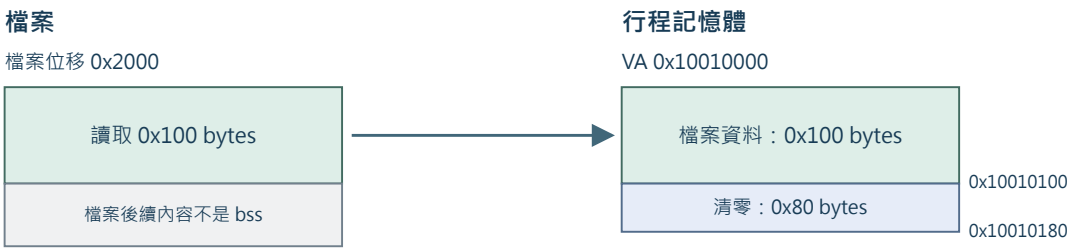
Static link把選取的library code合併進executable。Dynamic link則在process建立時或首次需要時，把reference綁到shared object的實際位址。不同process可以把同一唯讀code頁映到不同VA，共享的是physical backing，不表示可寫global data也必須共享。

用抽象GOT slot例，code從固定可達的slot讀取function pointer。Load前slot有待修補狀態；loader找到foo=0x70001234後填入slot；之後的indirect call取該值。Lazy binding可先導向resolver再更新slot，但實際stub/register規則依ABI，不把此抽象當作MIPS或x86的完整PLT指令。

ASLR讓base改變，position-independent code使用相對位址或間接表減少text修補。仍需正確symbol版本與呼叫ABI，載入相同名稱不保證介面相容。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.4 Loader 的 file size 與 memory size



memory size = 0x180；不代表載入器必須先逐 byte 複製。

明定一個可寫segment：file offset0x2000、VA0x10010000、file size0x100、memory size0x180，page size0x1000。Loader從檔案映入首0x100 bytes，並把後0x80 bytes清零；不把檔案後面不相關的bytes當作bss。

Segment與section不同。Linker可把多個sections合進一個具共同權限的loadable segment；loader主要依可執行格式的load資訊建立映射。Text可設R-X，data設RW-，未映射guard page捕捉部分越界。

Runtime entry先建立執行環境並呼叫main，因此main不是process的第一條instruction。實際ELF、PIE與interpreter流程依OS，不能只依section名稱推知全部runtime權限。

B.5 位址、byte order 與對齊

32-bit值0x12345678放在位址0x100：big-endian bytes為12、34、56、78；little-endian為78、56、34、12。Endianness改變byte位置，不改變register中數值，也不把指令的欄位bits左右顛倒。

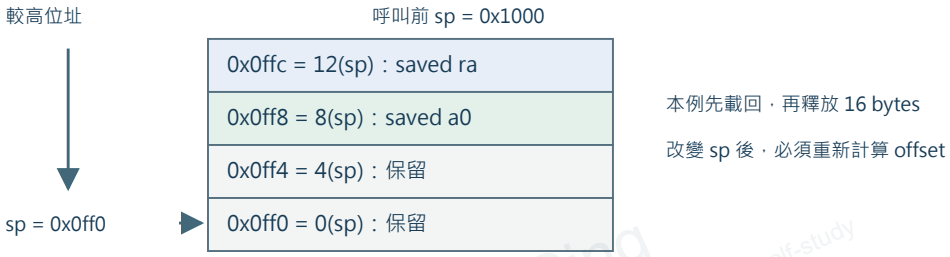
Strict-alignment MIPS的lw要求address低2 bits為0；0x102不能直接作lw。Packet序列化要固定欄位寬度、byte order及padding；native struct的memory dump不是跨機器協定。

Stack與heap的成長方向、mapping與guard由ABI/OS決定，典型示意不是所有process的唯一layout。

B.6 Caller 與 callee 保存責任

Caller把前四個整數arguments放a0–a3，常以v0/v1取結果。t-registers通常caller-saved，s-registers通常callee-saved；callee若改s0，return前須還原incoming s0。Caller若call後還要舊a0，需自己保存。

16-byte frame：地址與保存位置



本章遞迴例使用16-byte簡化教學frame，不宣稱完整o32 non-leaf frame。完整ABI還可能要求outgoing argument area、特定alignment及其他保存欄位。只符合ISA、卻不符合呼叫慣例的函式，仍可能破壞其他編譯器產生的caller。

sum_to 的完整遞迴程式

明定0<=n且和不溢出signed32-bit，sp初始aligned且空間足夠。本程式使用addu作模2^32相加，輸入界限確保同時等於數學答案。

```
sum_to:
  addiu $sp,$sp,-16
  sw    $ra,12($sp)
  sw    $a0,8($sp)
  beq   $a0,$zero,base
  nop
  addiu $a0,$a0,-1
  jal   sum_to
  nop
  lw    $t0,8($sp)
  addu  $v0,$v0,$t0
  j     done
  nop
base:
  addu  $v0,$zero,$zero
done:
  lw    $ra,12($sp)
  addiu $sp,$sp,16
  jr    $ra
  nop
```

每條控制轉移後都有slot。遞迴返回後重新從本層frame讀n，不能假設a0仍是原值，也不能把遞迴呼叫前的t0當作callee必保留值。

遞迴的實際 frames 與返回值

以初始sp=0x1000呼叫sum_to(3)，base case也建立frame。

n	本層sp	saved n位址	saved ra位址	返回v0
3	0x0ff0	0x0ff8	0x0ffc	6
2	0x0fe0	0x0fe8	0x0fec	3
1	0x0fd0	0x0fd8	0x0fdc	1
0	0x0fc0	0x0fc8	0x0fcc	0

返回由最內層依序做0、0+1=1、1+2=3、3+3=6，sp回到0x1000。最大同時4 frames，共64 bytes。本例各層先用本層sp對應的offset載回ra，再釋放frame；若先改sp，仍可用重新計算的地址讀取，但不能繼續盲用舊offset。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

遞迴正確性的歸納

令函式契約為：輸入n滿足範圍時，return有v0=n(n+1)/2、sp與callee-saved registers恢復。n=0時設定v0=0，epilogue取回ra/sp，符合契約。

對n>0，假設sum_to(n−1)符合契約。呼叫前把本層n與ra保存在自身frame，child的frame更低且不重疊；child返回後sp回到本層值，取回n並相加得(n−1)n/2+n=n(n+1)/2，最後還原ra/sp。n每次減1且有下界0，故終止。

此證明依賴stack容量與不溢出前提。無限遞迴、negative n或stack overflow不在契約內，不能只用已跑過n=3當作一般證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.7 Exception 不受一般 call ABI 限制

同步exception由當前指令引起，如overflow、address fault或syscall；interrupt來自timer或device等外部事件。Handler可在任何程式位置打斷執行，不能只保存callee-saved registers，因為被中斷程式也可能正使用caller-saved registers。

硬體保存或提供恢復所需的PC與status，軟體保存自己會改寫的registers。Handler需判斷cause、處理/清除必要pending source，再依可恢復語意返回。未清裝置原因就return，可能立即再次中斷；nested handler也需獨立保存狀態，避免覆寫outer context。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

EPC 與 branch delay 的兩種情形

非delay-slot的可恢復fault：Cause.BD=0，EPC指向faulting instruction。若修好page mapping並重試load，通常回到該EPC；若已完成一個syscall服務，則在明定政策下由下一條繼續。

若branch在0x00400020，其delay slot的lw在0x00400024發生address fault，則BD=1、EPC=0x00400020。EPC指向branch，不是lw。單純EPC+4會直接跳到slot且遺失branch語意；handler須保留/重建branch決策與slot狀態，或採可正確重新執行的處理方式。

Precise exception要求較舊指令已完成、faulting及較年輕指令未提交不應有的effects。外部interrupt優先權不能讓較年輕同步fault越過較舊指令的程式順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.8 MMIO receiver 的 polling 例

以下是明定教學device，不宣稱所有SPIM版本固定此位址。Receiver control=0xffff0000、data=0xffff0004，control bit0=ready；讀data消耗一個byte並清ready。MMIO映射uncached，平台保證本序列所需I/O順序。

```
lui   $t0,0xffff
poll:
lw    $t1,0($t0)
andi  $t1,$t1,1
beq   $t1,$zero,poll
nop
lw    $v0,4($t0)
andi  $v0,$v0,0xff
```

若control依序回傳0、0、1，loop讀control三次，只在第三次後讀data一次。把data load提前到branch delay slot會在not-ready時也讀取，改變裝置狀態，故slot保留nop。

B.9 SPIM syscall 的可執行語意

在SPIM教學服務中，print-int常用v0=1、a0=整數；print-string用v0=4、a0=NUL結尾字串位址；exit用v0=10。服務編號是simulator介面，不是ISA opcode或跨OS通用編號。

```
addiu $v0,$zero,1
addiu $a0,$zero,42
syscall
addiu $v0,$zero,10
syscall
```

此處syscall不在delay slot。Simulator可能直接實作服務，不等同真OS執行一份相同的handler。觀察到輸出42可核對此程式，不足以證明真實MIPS系統的exception vector或kernel ABI相同。

B.10 指令語意的查核表

類別/例	位元運算與重要條件
add / addu	同樣低32-bit和；add可因signed overflow trap，addu不trap
addiu	immediate sign-extend，不因signed overflow trap
andi / ori	immediate zero-extend
slt / sltu	相同比特以signed/unsigned比較，可得不同結果
lb / lbu	讀8-bit後分別sign/zero extend
lw / sw	base+signext immediate；strict model要求word對齊
beq / bne	比較GPR，target=PC+4+(signext imm<<2)，執行slot
j / jal	拼接PC+4高bits；jal另外寫ra=PC+8
jr	register target，仍有slot

例如bit pattern0xffffffff，slt把它解成−1，sltu解成4294967295。Signedness由使用的operation與資料契約決定，不是bits自己帶有signed標記。

B.11 從錯誤位址回到來源

Fault位址若落在錯誤function，先查symbol resolution與link map；若constant只差0x10000，核對low sign extension及HI/LO配對；若call返回後register錯，核對ABI與delay slot；若bytes顛倒，核對序列化及endianness。

Assembler listing可顯示source、offset與word；object dump顯示symbols/relocations；link map顯示最終section位置；execution trace顯示PC與register變化。這些證據定位不同邊界，不能用「可能compiler bug」取代具體欄位核對。

B.12 練習的模型邊界

本章的兩遍assembler、relocation、stack及MMIO例都先固定版本與語意，再推導結果。練習需要重算變更layout、低半部負值、archive相依與異常返回的影響，不能只背一組hex答案。

完整ISA還含floating-point、coprocessor與其他system instructions。查原書reference時應分清machine instruction與pseudo，並同時讀語意、例外、對齊及delay-slot限制，不能只查opcode一欄。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料

Patterson與Hennessy，*Computer Organization and Design: The Hardware/Software Interface*，Revised Fourth Edition，Appendix B。B.2–B.9印刷B-10至B-44（PDF實體815–849）；B.3.1連結圖在印刷B-19（PDF824）。B.10為指令reference，印刷B-45至B-80（PDF850–885）。

本文object layout、數值、遞迴程式與練習為明定模型的獨立例子。需要完整平台ABI時，須再查該系統規格，不能把16-byte教學frame當作完整o32 non-leaf配置。

參考資料與署名

David A. Patterson、John L. Hennessy，Computer Organization and Design，Revised Fourth Edition；Lin Cing，《計算機組織》手寫筆記；計算機組織與結構（CSO）課程投影片。

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study